Project FieldDay Software Development Plan

ECE 49595: OSS SD I
Section: 070

Team Members:

Ethan Ottinger: ejotting@purdue.edu

Gabriel Ogbalor: gogbalor@purdue.edu

Gustav Karlsson: pkarlsso@purdue.edu

Uma Srinivas: usriniva@purdue.edu

Project Overview

The purpose of our project is to create a means with which athletes with unfulfilled passions for sports can find, plan, and play local recreational sports. There is a noticeable lack of local sporting communities, so this mobile application aims to be able to form one through grouping people that play the same sport with similar skill levels.

Scope

In:

- Feature to create and join sports sessions in a defined geographical area
- Recommendation feature to group athletes based on friends, sports, and skill level
- Feature to assess user credibility by giving each user a social credibility rating
- Feature to penalize poor behavior/skipping sessions through a user suspension system
- Mobile application for ease of use

Under Evaluation:

- Social messaging or chat feature external to groups
- Private/friends-only restricted session creation feature
- External skill verification feature (through already established sports ranking metrics)
- Customizable privacy settings that restrict the information others can view

Out:

- A learn-how-to-play-a-sport training session
- Venue/equipment reservation/monitoring
- Website

Assumptions:

- Users will report/estimate skill level honestly
- Users are motivated to make new connections and play with new/different people
- Users will show up to scheduled sessions

Constraints:

- Recommendation engine is limited by data constraints and training latency
- Users will only benefit from other users using the application
- Reimbursement budget and project due date

A feature to create and join sports sessions is the core purpose of our app, hence its inclusion in the “In” category. Additionally, we will need a recommendation algorithm so that people can meet and find other similar users to create a local sports community online. Furthermore, users need to be held accountable for the sake of other users’ time and the usability of our app, so we decided to include user credibility as well as penalties for skipping out on sessions that they signed up for. Finally, we are committed to developing FieldDay exclusively as a native mobile

application (React Native) to support the platform's core need for on-the-go, real-time usage. A mobile app provides the most performant and reliable integration with device-level location services necessary for geospatial matchmaking. Crucially, a native application maximizes convenience, eliminating the friction of using a browser and URL, which is vital for users who need to quickly join sessions while moving across different areas. This choice also ensures reliable, high-priority push notifications for real-time updates and user accountability.

We are still debating whether to include a social messaging feature, especially when it comes to communication outside of a group. Messaging within a group is almost mandatory, but an ability to chat outside of groups could possibly help users meet other users. Additionally, we are assessing if the ability to create private sessions fulfills the purpose of our app of creating a local community that wouldn't otherwise be possible. Private sessions would otherwise be possible, but the feature would still have some practical use cases. We are also considering external skill verification, as many sports already have "ratings" of players that have played that sport competitively. However, it might only be useful/possible for higher-end players. Finally, privacy is a big deal to a lot of people so we are considering what privacy features we should include, and if they should be customizable by the user.

Our app is not for beginners to learn how to play. We will provide a means with which beginners can connect with each other and learn, but a proctored training session would require resources we do not have. We will also not link our app to venue/equipment reservation sites, as that is up to the user. Ideally, local parks will be used as venues and the athletes own the required equipment. And as previously stated, we will not be constructing a corresponding website.

Our assumptions are consistent with the target user. Users should want to report skill accurately to make the competition as fair, and therefore fun, as possible. Users will want to find others to play with because our target audience has a lack of peers that are willing to play the sport with them. There is really no point in using the app without exploring and finding others to play with. Finally, we assume that users will be persuaded to show up to their scheduled sessions. If we enact a credibility system and a suspension system, users being able to continue using the app will be based on their ability to be honest and show up to sessions.

The major constraint of our project is the limitations introduced by the recommendation algorithm. Any imperfections in it will render the app useless, as users will not be able to find similar athletes. Another constraint that requires one of the previous assumptions is that users are what enable this app to be beneficial to use. Our app is constrained by the group of users actively using the app. Finally, a logical constraint on our project is the due date and $400 budget. We cannot spend years developing this app, nor can we use expensive resources to make our app as premium as possible. We need to determine an application that has maximal usage for the budget and due date.

Requirements Elicitation and Prioritization

Through the usage of the MoSCow method, we outlined the different requirements for our project as well as the specific things we do *not* intend on implementing as features. For example, it helped us realize the exact purpose of our application is an ability to create and join sporting sessions. It can be easy to get caught up in the could or should haves, such as creating a skill verification service, but MoSCow helped us prioritize what matters more. Additionally, ranking the various features we would like to have by considering whether they are a core functionality of the concept or simply a desirable addition will make it easier for us to keep track of what is a priority to work on and what can be removed/skipped should we find it too difficult or too time consuming to implement. Additionally, MoSCow enables us to restrict the scope with won't haves, as we will not have any venue reservation or staff. This is important to recognize early so that there is an ability to plan to compensate for what we won't have with some should have features. For example, penalties for bad ratings could act in place of regulatory staff. Going over the stakeholders was however less useful, given that we do not have many stakeholders involved in the project, being just us (the developers) and the end-users. Overall, I would not say that the MoSCow method is going to majorly alter how we approach the process of developing the app, but rather compliment it and assist us in our approach to doing so.

Software Requirements

<u>System Requirements defined in Project proposal:</u>

SYS-1: Scheduling feature that enables joining/viewing groups

SYS-2: Interactive social feature where users can chat/add friends

SYS-3: Profile customization feature for users to set preferences/skill levels

SYS-4: User review feature for rating a user's credibility

SYS-5: Recommendation algorithm that pairs athletes with relevant sessions

<u>Functional Requirements:</u>

ID: FR-1

Title: Session Creation and Scheduling

Statement: When an authenticated user initiates session creation, the system shall allow the user to specify the sport type, date and time, geographic location, maximum number of participants, and required skill level (1 through 3), and shall publish the session as available to other users.

Rationale: Without the ability to create structured sport sessions with metadata like location, time, sport, and skill level, no other feature in the app can function, as session creation is the bare foundation of the FieldDay app.

Test Method: Create 25 sessions with varying sport types, locations, times, skill levels, and participant limits. Verify each session appears in the session discovery list. Confirm all specified metadata (sport type, date/time, location, max participants, skill level) is accurately stored in MongoDB and correctly rendered to other users browsing sessions. Attempt to create a session with missing required fields (e.g., no sport type, no location) and verify the system rejects the submission with a descriptive error message. Log any failures and errors that occur. The success metric for this test is 100% among sessions.

Supporting Context: Sport Type refers to a sport selected from the list of sports in the database such as basketball, pickleball, volleyball etc. Skill level uses a 3-tier scale of level 1= no prior experience, level 2 = some experience playing sport but not on any organized level and level 3 = experience with sport at an organized level

Trace: SYS-1

Priority: Must Have

ID: FR-2

Title: Session Discovery and Joining

Statement: The system shall display a filterable list of available sport sessions based on the user's geographic location within a configurable radius (default 25 miles), sport type, and skill level, and when a user selects to join a session that has not reached its participant limit, the system shall add the user to the session roster and update the participant count in real time, under 250ms for API latency.

Rationale: Discovery and joining are the core actions of the FieldDay application. Location-based filtering ensures users only see sessions they could realistically attend, sport and skill filters narrow results to relevant matches, and real-time roster updates prevent overbooking of sessions.

Test Method: Generate 50 different sessions distributed across various geographical locations, sport types, skill levels and verify that a user in a given location sees only sessions within their configured radius. For each session, confirm that the user slot is incremented and once it reaches a defined session limit no joining is possible. Log any errors and failures. Verify API latency across geographical session joining. The success metric for this test is for over 95% of session requests to have an API latency under 250ms.

Supporting Context: A configurable radius means the user can define and adjust their search distance. It will leverage MongoDB's geoNear pipeline and requires GeoJSON-indexes session location data. Users viewing the same session see the updated participant count in real time. Sessions that have passed their scheduled end time are automatically excluded from discovery results

Trace: SYS-1, SYS-5

Priority: Must Have

ID: FR-3

Title: User Profile and Preferences

Statement: The system shall allow users to create and edit a profile that includes their display name, profile photo, preferred sports, self-assessed skill level (1 through 3) for each selected sport, weekly availability schedule, and a short bio.

Rationale: Without detailed sport preferences, skill levels, and availability data, recommendations become generic and session matching becomes inaccurate, undermining the app's core value proposition of connecting similarly-skilled athletes. The profile also serves as a user's identity within the community, building trust for in-person meetups

Test Method: Create a new account and verify the system prompts the user to complete profile setup during onboarding. Set preferences for 5 sports with varying skill levels (e.g., basketball at Level 3, tennis at Level 1), upload a profile photo, set a weekly availability schedule with multiple time blocks, and add a bio. Log out and back in; verify all data persists correctly. Edit the profile to change a sport's skill level from 2 to 3, verify the change reflects within 2 seconds,

and verify that recommended sessions (FR-5) update to reflect the new skill level on the next refresh cycle. Attempt to save a profile with no sports selected and verify the system requires at least one sport. The success metric for this test is 100% across all new accounts created.

Supporting Context: Weekly availability schedule is a set of time blocks (day of week + start time + end time) indicating general availability.

Trace: SYS-3

Priority: Must Have

ID: FR-4

Title: User Review and Social Rating System

Statement: After a scheduled sports session has ended, the system shall prompt each participant to rate and review every other participant on a scale of 1 to 5, and shall update each rated user's aggregate social rating as a continuous average. A score of 1 will be given to users that do not attend the session. There will be an option to add another user as a friend when rating said user.

Rationale: The social rating system incentivizes honest skill-level reporting, reliable attendance, and good sportsmanship by making each user's track record visible to the community. A user who repeatedly no-shows or misrepresents their ability will accumulate low ratings that signal unreliability.

Test Method: Complete a test session with 6 participants. Verify all 6 receive a rating prompt once the session ends. Have all the users submit ratings for one another and verify each user's social rating updates publicly to the correct rolling average. Verify a user cannot rate themselves. Verify a user cannot rate participants of a session they did not join. Verify that friend requests are sent to users that opted in to adding the user as a friend while rating. Verify a user cannot submit multiple ratings for the same participant in the same session. Log any errors and failures

Supporting Context: Social rating is an aggregate rating that a user has, based on the ratings the user has received across all sessions they have participated in. The social rating is on a scale of 1 through 5.

Trace: SYS - 4

Priority: Must Have

ID: FR-5

Title: Session Recommendation System

Statement: The system shall provide each active user with a ranked list of at least 5 recommended sport sessions, generated by the Shaped AI recommendation and ranking engine based on the user's sport preferences, skill level, geographic location, schedule availability, and historical interaction data (session views, sign-ups, and attendance), refreshed at least once every 24 hours.

Rationale: Shaped AI's machine-learning models learn from actual user behavior over time, delivering increasingly personalized suggestions that improve the chances of a user finding a great session and keeps users engaged with the platform.

Test Method: Create 10 user profiles with distinct sport preferences, skill levels, and locations. Seed the database with 200 sessions of varying sports, locations, times, and skill levels. Call the Shaped AI ranking API endpoint for each user and verify each receives at least 5 session recommendations. Verify recommended sessions align with the user's preferred sports and are within a reasonable geographic radius. Simulate 50 interactions (views, sign-ups) for a test user heavily skewed toward basketball; verify that the next recommendation refresh ranks basketball sessions higher than other sports. Measure Shaped API response latency. API response latency should be under 250ms. Verify that a user with no interaction history still receives recommendations based on their profile preferences (cold-start scenario).

Supporting Context: Shaped AI is a third-party ML-based recommendation and ranking service. Data flows from the MongoDB database to Shaped via defined connectors. Shaped ingests three tables: a user table (location, top sports, social rating, skill level, schedule), a sessions table (active and scheduled sessions), and an interaction table (views, sign-ups, attendance, friend connections). Shaped trains ranking models using embeddings for features like sport preferences, schedules, and skill level, learning patterns from actual user interaction outcomes rather than relying on manually assigned weights.

Trace: SYS-5

Priority: Must Have

ID: FR-6

Title: Student Verification System

Statement: The system shall integrate with the SheerID verification API to verify a user's active enrollment status using their .edu email address and will show a student verified badge on their profiles.

Rationale: The student verification provides a layer of comfort and gives an avenue to users that are on college campuses looking to play with other students in their campuses, make new friends in a new place and grow new connections with fellow students at their institution

Test Method: Initiate student verification with a valid .edu email address. Verify the system sends a verification request to SheerID and, upon a successful response, displays a successful verification response. Test with an invalid or non-.edu email and verify the system displays a clear rejection message.Log any errors and failures.

Supporting Context: SheerID is a third-party identity verification service specializing in student verification and is used by many leading companies such as Spotify, Apple Music, Cursor, Google Gemini, etc to verify student statuses.

Trace: SYS-3, SYS04

Priority: Could Have

ID: FR-7

Title: Location Services

Statement: The system shall integrate with the Google Maps SDK to display available sport sessions as pins on an interactive map centered on the user's current geographic location, and allow users to adjust radius to filter session by distance, as well as show the locations of scheduled and on going sessions

Rationale: Location based matchmaking is an essential aspect of FieldDay as it allows users to filter sessions that they can realistically make it to and as well shows them where sessions are being held.

Test Method: Generate 20 sessions at known geographic coordinates and verify each session pin appears at the correct position on the map. Test with device location services disabled and verify the system prompts the user to enable location or enter a manual address as a fallback. Test with no sessions within the selected radius and verify the map displays an appropriate "No sessions nearby" message.

Supporting Context: Google Maps SDK refers to the Maps SDK for React Native, providing map rendering, geocoding , and directions. Session locations are map markers placed at each session's GeoJSON coordinates stored in MongoDB. Location slider is done using MongoDB's geoNEar operators.

Trace: SYS-1

Priority: Must Have

ID: FR-8

Title: Cool Down and Suspension System

Statement: If a user's aggregate social rating falls below 2, then the system shall automatically impose a 2-day cooldown period during which the user cannot create or join new sessions

Rationale: This will deter users from making other users uncomfortable and from unreliable behaviours such as skipping sessions they have signed up for. This keeps users accountable and builds trust and community among users.

Test Method: Set a test user's social rating to below 2 and verify they cannot join or create sessions for a cooldown period of 2 days. Verify that after two days the user can freely create and join sessions.

Supporting Context: The cool down period is app level logic and will not be implemented in the Shaped recommendation engine.

Trace: SYS-4

Priority: Should Have

ID: FR-9

Title: Session Attendance Confirmation

Statement: The system shall require participants to confirm their attendance within a 10-minute window beginning at the scheduled session start time, and this confirmation shall only be

successful if the user's mobile device location confirms they are within the defined session radius. Failure to confirm attendance will result in the user being automatically marked as absent and receiving a rating of 1 for that session.

Rationale: Attendance confirmation ensures accountability and validates the user's presence at the specified location, leveraging the app's core location services to prevent no-shows and maintain the credibility of user ratings. Thus enhancing user trust in the sessions they attend.

Test Method: Schedule a session. Have a test user attempt to confirm attendance: 1) before the 10-minute window, 2) after the 10-minute window, 3) within the window but outside the radius, and 4) within the window and inside the radius. Verify rejection for scenarios 1-3 with an appropriate message and successful confirmation only for scenario 4. Verify that the user in scenario 3 is automatically assigned a rating of 1.

Supporting Context: The geographical radius is the same one set during session creation (FR-1) and joining (FR-2). Location verification relies on the device's GPS and is handled by Location Services (FR-7).

Trace: SYS-1, SYS-4

Priority: Must Have

ID: FR-10

Title: Friend Management and User Reporting

Statement: The system shall allow a user to perform two social actions after a session: (1) send a Friend Request to another participant, which requires the recipient to accept to establish a bidirectional connection; and (2) Report a user, allowing the sender to provide a free-text comment explaining the reason for the report (e.g., poor conduct, safety concern).

Rationale: Providing accessible tools for friend connection facilitates the community-building goal of FieldDay. The reporting mechanism is a critical safety feature that allows users to flag inappropriate behavior and provides the necessary data to enforce community guidelines and/or law enforcement.

Test Method: Complete a test session. User A sends a Friend Request to User B. Verify User B receives a notification. User B accepts; verify both users are now friends. User A sends a Friend Request to User C. User C declines; verify no friendship is established. User A reports User D

with a comment "Abusive language." Verify the report is logged in the backend database for review and User D is not immediately notified.

Supporting Context: Friend requests will initiate a connection for private messaging and friend activity visibility. Reports and flags will be taken up on and escalated to law enforcement if necessary

Trace: SYS-2, SYS-4

Priority: Must Have

Non-Functional Requirements

ID: NFR-1

Title: Performance (API latency and requests speed)

Statement: GraphQL API requests to the MongoDB database and the backend server (for session creation and joining) should be completed within 250ms of the request and app rendering should occur within a second on devices with 4G or faster network speed.

Rationale: As field day is a real-time platform where users can instantly find and join live and scheduled sessions, slow API responses or long screen loads will frustrate users and lead to poor user experience with the app.

Test Method: Use a load testing tool to simulate 1000 concurrent users and issue API requests for session creations, session joining, and profile creations. Measure and record response times for all requests to determine API latency of less than 3 seconds and verify no crashes or failures due to load.

Supporting Context: Normal operating conditions assumes a production deployment on AWS EC2 with Elastic Load Balancing as defined in the technology stack. Concurrent users means users with active WebSocket connections or pending API requests at the same point in time

Trace: SYS-1, SYS-5

Priority: Must Have

ID: NFR-2

Title: Security and Data Privacy

Statement: The system shall encrypt all user data in transit using TLS 1.2, encrypt sensitive data using AES-256, authenticate users with authentication layer such Google OAuth, and enforce role-based access controls such that users can only read and modify their own personal data.

Rationale: FieldDay handles personally identifiable information (names, email addresses, profile photos, real-time geographic locations, availability schedules) and facilitates in-person meetups between strangers. A security breach could expose users to identity theft, stalking, or harassment. The app's entire value depends on users trusting it enough to show up and meet people in person.

Test Method: Use an SSL/TLS testing tool (e.g., Qualys SSL Labs) to scan the API endpoints and verify TLS is enforced with no support for deprecated protocols. Inspect the MongoDB cluster configuration and verify encryption at rest is enabled with AES-256. Authenticate as one user and attempt to access other user's profile data via direct GraphQL queries, and verify the system returns a 403 Forbidden response for each. Verify the user document schema in MongoDB contains no password field and only OAuth tokens.

Supporting Context: Google OAuth means users authenticate by signing in with their Google account. The system stores an OAuth refresh token and a Google user ID. Role-based access controls differentiate between regular users, verified students, and system administrators. Sensitive data includes email addresses, location history, chat message content, and social rating history. AES-256 is a cipher algorithm that encrypts and decrypts keys using a 256-bit key. TLS 1.2 is a standard security protocol used to encrypt data and ensure data privacy between client and server communications.

Trace: SYS-2, SYS-3

Priority: Must Have

ID: NFR-3

Title: Reliability and Availability

Statement: In the event of an individual EC2 instance failure, the system shall automatically route traffic to a healthy instance via AWS Elastic Load Balancing with no loss of committed data.

Rationale: Sports sessions are time-sensitive events that happen in the real world, The app being down unreliably will cause users to miss their planned events and sessions and reduce the overall trust in our app causing us to potentially lose users.

Test Method: Deploy the production environment on 3 EC2 instances behind an Elastic Load Balancer. Monitor continuously for 30 days using AWS CloudWatch health checks at 2 minute intervals, and calculate monthly uptime. Simulate an EC2 instance failure by terminating one instance and measure the time until the load balancer routes 100% of traffic to the remaining healthy instances. Verify that WebSocket connections automatically reconnect to the healthy instance and check MongoDB database to confirm no write operations and records were lost during the failover event.

Supporting Context: Uptime is the percentage of time that the system's health check endpoint returns successfully. AWS EC2 provides virtual servers for cloud compute and AWS elastic load balancing distributes traffic among EC2 instances.

Trace: SYS-1, SYS-2

Priority: Must Have

ID: NFR-4

Title: Usability

Statement: The system shall provide a user interface that enables a first-time user to create an account and complete their profile (including selecting at least one sport and skill level) within 5 button interactions from launch without requiring external documentation, tutorials, or assistance. For recurring users, from login to join session should be within 3 button interactions.

Rationale: A long sign-up process will make it difficult to gain users, which is a critical part for the app's functionality. The main provided service of the app is directly caused by the presence of users, and as such making the sign-up process and overall experience with the app as smooth as possible is necessary.

Test Method: Launching the app without an existing account and measuring the number of interactions required to join a sport session with other users.

Supporting Context: Each user will have to sign up with preferred sport(s) and skill level(s) in order for the app to function. This will be a one time "sign up." Our goal is to make it extremely

easy to get into the actual functionality of the app as a new user. Therefore, we made a button (or page) goal of less than 5. These would include a page for sports and skill levels, entering their name, enabling push notifications, and making a username (or email) and password. For recurring users, it should only take 3 button presses to join a group: one to view the calendar page of all open groups that match your preferences, another to inspect a particular group, and the final one being joining or requesting to join the group.

Trace: SYS-1, SYS-2, SYS-3

Priority: Must Have

Deliverables

Deliverable: A production-ready React Native mobile application published on the iOS App Store and Google Play Store that enables users to create and join local sport sessions, discover sessions on an interactive map, set up profiles with sport preferences and skill levels, rate other participants after sessions and receive session recommendations.

Requirements: FR-1 Session Creation and Scheduling, FR-2 Session Discovery and Joining, FR-3 User Profile and Preferences, FR-4 User Review and Social Rating System, FR-5 Session Recommendation System, FR-7 Location Services, NFR-1 Performance, NFR-4 Usability

Deliverable: A documentation package consisting of end-user documentation including a quick-start tutorial walking new users through account creation, profile setup, session discovery, and joining their first session, and developer documentation including instructions for running the app locally, API specifications (GraphQL schema docs), MongoDB schema definitions, C4 architecture diagrams (system context, container, and component levels), and a build/deployment guide covering Docker, AWS, and CI/CD pipeline usage.

Requirements: NFR-3 Reliability and Availability, NFR-4 Usability

Deliverable: A Safety Incident Report. This report will be a compiled record of all user-reported hazardous actions and system-triggered suspension events that occurred during sessions, providing necessary data for community enforcement and law enforcement escalation.

Requirements: FR-8 Cool Down and Suspension System, FR-10 Friend Management and User Reporting, NFR-2 Security and Data Privacy

~~Deliverable: A TypeScript/Express.js backend API server deployed on AWS EC2, providing a GraphQL API via Apollo Server for all client-server communication, WebSocket endpoints via Socket.IO for real-time chat and notifications, and REST integrations with external services (Shaped AI, Google Maps, SheerID, AWS SNS).~~

~~Requirements: FR-1 Session Creation and Scheduling, FR-2 Session Discovery and Joining, FR-4 User Review and Social Rating System, FR-5 Session Recommendation System, NFR-1 Performance, NFR-2 Security and Data Privacy, NFR-4 Reliability and Availability~~

~~Deliverable: A fully defined MongoDB database (hosted on MongoDB Atlas) with schemas and indexes for users, sport sessions, interactions, chat messages, friend relationships, social ratings, including GeoJSON geospatial indexes for location-based session queries and the data connectors required for the Shaped AI data pipeline.~~

~~Requirements: FR-1 Session Creation and Scheduling, FR-2 Session Discovery and Joining, FR-3 User Profile and Preferences, FR-4 User Review and Social Rating System, NFR-1 Performance, NFR-2 Security and Data Privacy~~

~~Deliverable: A Shaped AI integration package consisting of a YAML configuration file defining the data schema and embedding configuration (user table, sessions table, interaction table), MongoDB-to-Shaped data connectors, and the API integration code for requesting and caching ranked session recommendations from the Shaped ranking endpoint.~~

~~Requirements: FR-5 Session Recommendation System~~

~~Deliverable: AWS cloud deployment configuration files including EC2 instance provisioning, Elastic Load Balancer setup, AWS SNS topic and subscription definitions for push notifications, and environment variable configurations for all external service keys (Google OAuth, Google Maps, SheerID, Shaped AI).~~

~~Requirements: FR-5 Session Recommendation System, FR-7 Location Services, NFR-1 Performance, NFR-3 Reliability and Availability~~

~~Deliverable: Docker and containerization files (Dockerfiles) for the backend API server and any supporting services, enabling reproducible local development builds, and consistent staging/production deployments.~~

~~Requirements: FR-1 Session Creation and Scheduling, NFR-4 Reliability and Availability~~

~~Deliverable: A GitHub Actions CI/CD pipeline providing unit and integration test execution (Jest), Docker image builds, and deployment to AWS EC2~~

~~Requirements: NFR-1 Performance, NFR-2 Security and Data Privacy, NFR-3 Reliability and Availability~~

Standards

Information Security Management

URL: https://www.iso.org/isoiec-27001-information-security.html

- Applicability: This standard is crucial for handling the risks linked to our user database, especially in safeguarding sensitive information such as student verification statuses and individual skill levels.
- Impact on Design/Code: We will apply the principle of least privilege to all AWS resources and utilize strong encryption and authentication protocols to ensure that private data is accessible only to authorized users.
- Compliance Plan: We will ensure compliance by conducting audits of the authorization layers on our Apollo GraphQL server and by verifying that all MongoDB records are securely stored.

Interaction Capability

URL: https://www.iso.org/standard/78174.html

- Applicability: This standard guarantees that users can easily and efficiently reach their objectives, like filtering sports sessions based on skill level and location, while keeping the process simple and streamlined.
- Impact on Design/Code: Our design will prioritize a straightforward user interface with minimal transitions to keep athletes engaged while they are on the move.
- Compliance Plan: Usability will be assessed by counting the clicks needed for onboarding and conducting user tests on the interactive social features.

Functional Suitability

URL: https://www.iso.org/standard/78174.html

- Applicability: Functional suitability guarantees that the application delivers the necessary features to satisfy user requirements, including session creation, joining, and the recommendation engine.
- Impact on Design/Code: It ensures a precise connection between the code and our functional requirements, enabling the recommendation algorithm to effectively incorporate sports preferences and geographical information when suggesting sessions.
- Compliance Plan: We will utilize full coverage test cases with Jest to ensure that every core feature, including location-based discovery and the suspension system, operates precisely as detailed in the proposal.

References:

[1]ISO, “ISO/IEC 27001 Information security management,” *ISO*, 2022. https://www.iso.org/isoiec-27001-information-security.html

[2]O. for, “ISO 16055:2021,” *ISO*, 2021. https://www.iso.org/standard/78174.html (accessed Mar. 09, 2026).

[3]O. for, “ISO 16055:2021,” *ISO*, 2021. https://www.iso.org/standard/78174.html (accessed Mar. 09, 2026).

Development Methodology

We have selected the Scrum agile framework to manage the development of FieldDay. We chose Scrum because its fixed-length iterations (sprints) enforce a regular, structured cadence that is essential for a project with complex, real-time features like geospatial matching and the Shaped AI recommendation engine. Scrum's core strength lies in its feedback loop, allowing the team to deliver working software increments at the end of each sprint for review, enabling corrections before significant effort is wasted. This iterative approach is crucial for adapting to the evolving requirements typical of a complex software product.

Verification and Validation Plan

Link to Verification and Validation Plan:
https://docs.google.com/document/d/14doR2u9QAM718IMXEhuzYEnfhys4ccF00G4mqgsVqaE/edit?usp=sharing

Gantt Chart

Link to Gantt Chart:
https://drive.google.com/file/d/1EI7Jq_IFB8iK-moVlafbsczXywLH1HDQ/view?usp=sharing